

1.1 Scenario

Student Crowdfunding Platform

The platform allows student founders to submit ideas/campaigns and allows registered backers to discover campaigns, select reward tiers, contribute funds, track milestones, and receive fund disbursements based on verified project progress.

1.2 Functional Requirements

Req ID	Functional Requirement
FR-001	Student founders shall be able to submit their ideas/campaigns onto the platform.
FR-002	Backers shall be able to discover campaigns and view campaign descriptions, student history, and reward tiers.
FR-003	Backers shall be able to view/select reward tiers and make funding contributions.
FR-004	Backers shall be able to track campaign progress, milestones, and fund-release status.
FR-005	The system shall release funds to student ventures in tranches after predetermined milestones are verified.

1.3 Non-Functional Requirements

Req ID	Non-Functional Requirement
NFR-001	User information and financial transaction data must be securely stored and protected from unauthorized access.
NFR-002	Payment processing must comply with PCI-DSS guidelines and use tokenized payment authorization while maintaining acceptable performance under peak load.

1.4 Main Challenges

A. Performance

The platform may have many users browsing campaigns and making contributions simultaneously.

Important performance concerns:

- Campaign pages should load quickly.
- Payment processing should have low latency.
- The system should handle peak contribution periods.
- Fund disbursement should not unnecessarily slow down other platform operations.

B. Usability

The platform has two major types of users:

- **Student founders**
- **Campaign backers**

Therefore:

- Student founders need a simple way to submit ideas.
- Backers need to easily discover campaigns.
- Reward tiers should be clearly displayed.
- Funding progress and milestone status should be easy to understand.

C. Security

Security is the **most important challenge** because the platform handles financial transactions.

The system needs to:

- Protect user credentials.
- Protect financial transaction information.
- Prevent unauthorized access.
- Secure payment processing.
- Use tokenized payment authorization.
- Follow PCI-DSS requirements.

Step 2: Architectural Style Analysis

2.1 Layered Architecture

Structure

The system is divided into separate horizontal layers. For our crowdfunding platform, the layers can be:

- **Presentation Layer** – Student founder/backer interface
- **Business Logic Layer** – Campaign, reward, payment and milestone processing
- **Data Layer** – User, campaign and transaction data

Advantages

1. **Clear separation of concerns**
Each layer has a specific responsibility, making the system easier to understand and maintain.
2. **Easy development and maintenance**
Changes in the user interface can generally be handled separately from business logic and data storage.
3. **Centralized data management**
The system can maintain campaign, user and transaction data in a structured data layer.

Disadvantages

1. **Limited independent scalability**
If campaign discovery receives very high traffic, scaling the entire application may be necessary instead of scaling only that functionality.
2. **Tight dependency between layers**
Changes may need to pass through multiple layers.

3. **Performance overhead**

Requests may have to pass through several layers before reaching the required data or functionality.

The handout similarly identifies **clear separation of concerns** and suitability for traditional applications as advantages, while noting performance overhead, tighter coupling, and difficulty scaling individual components as disadvantages.

Suitability for our platform

Good, because the crowdfunding platform has clear presentation, business, and data responsibilities. However, independent scaling of payment and campaign services is more difficult.

2.2 Microservices Architecture

Structure

The system is divided into multiple small, independently deployable services.

Each service handles a specific business responsibility.

Advantages

1. **Independent scaling**

Campaign discovery and payment processing can be scaled independently according to their workloads.

2. **Fault isolation**

A failure in one service does not necessarily bring down the entire platform.

3. **Independent deployment**

Individual services can be modified or deployed without redeploying the entire application.

4. **Good modularity**

Functions such as payment, campaign management and milestone verification are clearly separated.

The handout specifically lists **independent scaling, fault isolation, technology diversity, and independent deployment** as advantages.

Disadvantages

1. **Operational complexity**
There are multiple services to deploy, monitor and maintain.
2. **Network latency**
Services communicate over a network, which can introduce additional latency.
3. **Data consistency challenges**
Maintaining consistent information across different services can be more difficult.

Suitability for our platform

Very good / Best suited, because our crowdfunding platform naturally contains independent functionalities such as:

- Campaign management
 - Campaign discovery
 - Reward and contribution processing
 - Payment processing
 - Milestone verification
 - Fund disbursement
-

2.3 Client-Server Architecture

The student founder and campaign backer act as clients, while the central server manages the platform's functionality and data.

Advantages

1. **Centralized control**
Business logic and data can be managed from a central server.
2. **Simple deployment**
The architecture is relatively straightforward to deploy and maintain.
3. **Easy data consistency**
Data is managed centrally, making consistency easier to maintain.

The handout identifies **centralized control, simple deployment, and easy data consistency** as the main advantages.

Disadvantages

1. **Single point of failure**
If the central server fails, the entire platform may become unavailable.

2. **Scalability bottleneck**
A large number of users accessing the server simultaneously can overload it.
3. **Limited fault tolerance**
Failure of the central server can affect multiple functions simultaneously.

Suitability for our platform

Moderate, because it is simple and centralized, but the platform may need to handle many simultaneous campaign views and financial transactions.

Step 3: Architecture Selection & Component Identification

3.1 Architecture Selection

Selected Architecture: Microservices Architecture

Microservices is suitable because the platform has independent functionalities such as campaign management, payments, milestone verification, and fund disbursement. Each service can be independently developed, deployed, scaled, and maintained.

3.2 Component Identification

No.	Component	Responsibility
1	Campaign & Contribution Manager	Manage campaigns, rewards, and contributions
2	Payment Service	Process and authorize payments
3	Campaign Discovery Service	Browse and view campaigns
4	Milestone Verification Service	Track and verify project milestones

3.3 Component Relationships

**Campaign Discovery Service → Campaign & Contribution Manager → Payment Service
→ Milestone Verification Service → Fund Disbursement Service**

These five components cover the major functional requirements of the Student Crowdfunding Platform